

Ampliación Django: productos por usuario y control de permisos

Objetivo

Modificar el proyecto para que cada **Producto** tenga un usuario propietario.

Con esta ampliación:

- cualquier usuario podrá ver el listado general de productos;
 - solo usuarios autenticados podrán crear productos;
 - cada producto quedará asociado automáticamente al usuario que lo crea;
 - un usuario solo podrá editar sus propios productos;
 - un usuario solo podrá eliminar sus propios productos;
 - cada usuario tendrá una página llamada **Mis productos**.
-

1. Modificar el modelo **Producto**

Abrir:

productos/models.py

Importar el modelo **User**:

```
from django.contrib.auth.models import User
```

Modificar **Producto**:

```
from django.db import models
from django.contrib.auth.models import User
```

```
class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True)
    fecha_creacion = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nombre
```

```
class Producto(models.Model):
    nombre = models.CharField(max_length=150)

    descripcion = models.TextField(blank=True)

    precio = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )

    categoria = models.ForeignKey(
        Categoria,
        on_delete=models.CASCADE,
        related_name="productos"
    )

    usuario = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name="productos"
    )

    fecha_creacion = models.DateTimeField(
        auto_now_add=True
    )

    def __str__(self):
        return self.nombre
```

Ahora la relación será:

Usuario 1 ----- N Producto

Categoria 1 ----- N Producto

Esto significa que cada producto tendrá:

- una categoría;
- un usuario propietario.

2. Crear las migraciones

Ejecutar:

```
python manage.py makemigrations
```

Si ya existen productos en la base de datos, Django puede preguntar qué valor debe asignar al nuevo campo `usuario`.

Durante una práctica sencilla también se puede borrar la base de datos y volver a crearla si no existen datos importantes.

Después:

```
python manage.py migrate
```

3. Modificar `ProductoForm`

No queremos que el usuario pueda elegir manualmente el propietario del producto.

Por tanto, `usuario` no debe aparecer en el formulario.

Archivo:

`productos/forms.py`

El formulario seguirá así:

```
class ProductoForm(
    BootstrapFormMixin,
    forms.ModelForm
):
```

```
    class Meta:
```

```
        model = Producto
```

```
        fields = [
            "nombre",
            "descripcion",
            "precio",
            "categoria",
        ]
```

```
    def __init__(
        self,
        *args,
        **kwargs
    ):
```

```
super().__init__(
    *args,
    **kwargs
)

self.aplicar_bootstrap()
```

Observa que no aparece:

"usuario"

El propietario se asignará automáticamente en la vista.

4. Crear producto asignando el usuario

Modificar la vista:

```
@login_required
def producto_crear(request):

    if request.method == "POST":

        form = ProductoForm(request.POST)

        if form.is_valid():

            producto = form.save(
                commit=False
            )

            producto.usuario = request.user

            producto.save()

            messages.success(
                request,
                "Producto creado correctamente."
            )

            return redirect(
                "mis_productos"
            )

    else:
```

```
form = ProductoForm()

return render(
    request,
    "productos/formulario.html",
    {
        "form": form,
        "titulo": "Nuevo producto",
    }
)
```

La parte importante es:

```
producto = form.save(commit=False)
```

```
producto.usuario = request.user
```

```
producto.save()
```

Con:

```
commit=False
```

Django crea el objeto en memoria pero todavía no lo guarda.

Después asignamos:

```
producto.usuario = request.user
```

y finalmente:

```
producto.save()
```

5. Listado general

El listado general puede mostrar todos los productos.

```
def producto_lista(request):
```

```
    productos = Producto.objects.select_related(
        "categoria",
        "usuario"
    ).order_by("-fecha_creacion")
```

```
    return render(
        request,
        "productos/lista.html",
```

```
{
    "productos": productos
}
)
```

6. Crear la página "Mis productos"

Añadir una nueva vista:

```
@login_required
def mis_productos(request):

    productos = Producto.objects.filter(
        usuario=request.user
    ).select_related(
        "categoria"
    ).order_by(
        "-fecha_creacion"
    )

    return render(
        request,
        "productos/mis_productos.html",
        {
            "productos": productos
        }
    )
```

La consulta:

```
Producto.objects.filter(
    usuario=request.user
)
```

garantiza que solamente se obtengan los productos del usuario que ha iniciado sesión.

7. Proteger la edición

Antes teníamos:

```
producto = get_object_or_404(
    Producto,
    pk=pk
```

)

Ahora debemos comprobar también el propietario.

La vista quedará así:

```
@login_required
def producto_editar(request, pk):

    producto = get_object_or_404(
        Producto,
        pk=pk,
        usuario=request.user
    )

    if request.method == "POST":

        form = ProductoForm(
            request.POST,
            instance=producto
        )

        if form.is_valid():

            form.save()

            messages.success(
                request,
                "Producto actualizado correctamente."
            )

            return redirect(
                "mis_productos"
            )

        else:

            form = ProductoForm(
                instance=producto
            )

    return render(
        request,
        "productos/formulario.html",
        {
            "form": form,
            "titulo": "Editar producto",
        }
    )
```

```
)
```

La comprobación principal es:

```
get_object_or_404(
    Producto,
    pk=pk,
    usuario=request.user
)
```

Esto comprueba simultáneamente:

- que el producto exista;
 - que pertenezca al usuario actual.
-

8. Proteger la eliminación

Modificar:

```
@login_required
def producto_eliminar(request, pk):

    producto = get_object_or_404(
        Producto,
        pk=pk,
        usuario=request.user
    )

    if request.method == "POST":

        producto.delete()

        messages.success(
            request,
            "Producto eliminado correctamente."
        )

        return redirect(
            "mis_productos"
        )

    return render(
        request,
        "productos/eliminar.html",
        {
```



```
        "producto": producto
    }
)
```

De nuevo:

usuario=request.user

impide eliminar productos de otros usuarios.

9. Detalle del producto

El detalle puede ser público:

def producto_detalle(request, pk):

```
    producto = get_object_or_404(
        Producto.objects.select_related(
            "categoria",
            "usuario"
        ),
        pk=pk
    )

    return render(
        request,
        "productos/detalle.html",
        {
            "producto": producto
        }
    )
```

10. Actualizar las URLs

Archivo:

productos/urls.py

Añadir:

```
path(
    "mis-productos/",
    views.mis_productos,
```

```
    name="mis_productos"
),
```

La parte de productos puede quedar:

```
path(
    "productos/",
    views.producto_lista,
    name="producto_lista"
),
```

```
path(
    "mis-productos/",
    views.mis_productos,
    name="mis_productos"
),
```

```
path(
    "productos/nuevo/",
    views.producto_crear,
    name="producto_crear"
),
```

```
path(
    "productos/<int:pk>/",
    views.producto_detalle,
    name="producto_detalle"
),
```

```
path(
    "productos/<int:pk>/editar/",
    views.producto_editar,
    name="producto_editar"
),
```

```
path(
    "productos/<int:pk>/eliminar/",
    views.producto_eliminar,
    name="producto_eliminar"
),
```

Es recomendable colocar:

productos/nuevo/

antes de:

productos/<int:pk>/

11. Añadir "Mis productos" al menú

En:

templates/base.html

Dentro de:

```
{% if user.is_authenticated %}
```

añadir:

```
<li class="nav-item">

    <a
        class="nav-link"
        href="{% url 'mis_productos' %}"
    >
        Mis productos
    </a>

</li>
```

La navegación podría quedar:

Inicio
Productos
Categorías
Mis productos
Hola, juan
Cerrar sesión

12. Template `mis_productos.html`

Crear:

templates/productos/mis_productos.html

Código:

```
{% extends "base.html" %}

{% block title %}
Mis productos
{% endblock %}
```

```
{% block content %}
```

```
<div
  class="d-flex justify-content-between
  align-items-center mb-4"
>
```

```
<div>
```

```
<h1>
```

```
  Mis productos
```

```
</h1>
```

```
<p class="text-muted mb-0">
```

```
  Productos creados por
  {{ user.username }}.
```

```
</p>
```

```
</div>
```

```
<a
```

```
  href="{% url 'producto_crear' %}"
  class="btn btn-primary"
```

```
>
```

```
  + Nuevo producto
```

```
</a>
```

```
</div>
```

```
{% if productos %}
```

```
<div class="table-responsive">
```

```
<table
```

```
  class="table table-hover
  align-middle bg-white shadow-sm"
```

```
>
```

```
<thead>
```

```
<tr>
```

```
<th>
```

```
  Producto
```

```
</th>
```

```

        <th>
            Categoría
        </th>

        <th>
            Precio
        </th>

        <th>
            Fecha
        </th>

        <th>
            Acciones
        </th>

    </tr>

</thead>

<tbody>

    {% for producto in productos %}

        <tr>

            <td>

                {{ producto.nombre }}

            </td>

            <td>

                {{ producto.categoria.nombre }}

            </td>

            <td>

                {{ producto.precio }} €

            </td>

            <td>

                {{ producto.fecha_creacion|date:"d/m/Y H:i" }}

            </td>

```

</td>

<td>

<a
 href="{% url 'producto_detalle' producto.pk %}"
 class="btn btn-sm btn-outline-primary"
>
 Ver

<a
 href="{% url 'producto_editar' producto.pk %}"
 class="btn btn-sm btn-outline-secondary"
>
 Editar

<a
 href="{% url 'producto_eliminar' producto.pk %}"
 class="btn btn-sm btn-outline-danger"
>
 Eliminar

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% else %}

<div class="alert alert-info">

Todavía no has creado ningún producto.

</div>

<a
 href="{% url 'producto_crear' %}"
 class="btn btn-primary"

```
>  
  Crear mi primer producto  
</a>
```

```
{% endif %}
```

```
{% endblock %}
```

13. Mostrar propietario en el listado general

En:

templates/productos/lista.html

podemos mostrar:

```
<p class="text-muted">
```

Categoría:

```
{{ producto.categoria.nombre }}
```

```
</p>
```

```
<p class="text-muted">
```

Publicado por:

```
{{ producto.usuario.username }}
```

```
</p>
```

14. Mostrar botones solo al propietario

Aunque la vista ya está protegida, también debemos evitar mostrar botones incorrectos en la interfaz.

En el listado general:

```
{% if user == producto.usuario %}
```

```
<a
```

```
  href="{% url 'producto_editar' producto.pk %}"
```

```

        class="btn btn-outline-secondary btn-sm"
    >
        Editar
    </a>

    <a
        href="{% url 'producto_eliminar' producto.pk %}"
        class="btn btn-outline-danger btn-sm"
    >
        Eliminar
    </a>

{% endif %}

```

Esto mejora la interfaz.

Sin embargo, esta comprobación en HTML no sustituye a la seguridad de la vista.

La seguridad real sigue estando aquí:

```

producto = get_object_or_404(
    Producto,
    pk=pk,
    usuario=request.user
)

```

15. Detalle del producto con permisos

Modificar:

templates/productos/detalle.html

Ejemplo:

```

{% extends "base.html" %}

{% block content %}

<div class="card shadow-sm">

    <div class="card-body">

        <h1>

            {{ producto.nombre }}

```


</h1>

<p>

Categoría:

{{ producto.categoria.nombre }}

</p>

<p>

Precio:

{{ producto.precio }} €

</p>

<p>

Propietario:

{{ producto.usuario.username }}

</p>

<p>

Descripción:

</p>

<p>

{{ producto.descripcion }}

</p>

{% if user == producto.usuario %}

```
<hr>

<a
  href="{% url 'producto_editar' producto.pk %}"
  class="btn btn-secondary"
>
  Editar
</a>

<a
  href="{% url 'producto_eliminar' producto.pk %}"
  class="btn btn-danger"
>
  Eliminar
</a>

{% endif %}

</div>

</div>

{% endblock %}
```

16. ¿Qué ocurre si un usuario modifica la URL?

Supongamos:

Usuario Juan
Producto ID 5

Juan puede acceder a:

/productos/5/editar/

Ahora María inicia sesión e intenta escribir manualmente:

/productos/5/editar/

La vista ejecuta:

```
get_object_or_404(
    Producto,
    pk=5,
```

```
    usuario=request.user  
)
```

Django buscará:

Producto:
id = 5
usuario = María

Como el producto pertenece a Juan, no se encontrará.

Resultado:

404 Not Found

María no puede modificar el producto de Juan.

17. Alternativa mostrando error 403

También podemos comprobar los permisos manualmente.

Ejemplo:

```
from django.core.exceptions import PermissionDenied
```

Y después:

```
producto = get_object_or_404(  
    Producto,  
    pk=pk  
)
```

```
if producto.usuario != request.user:
```

```
    raise PermissionDenied
```

Esto devuelve:

403 Forbidden

Semánticamente:

404

significa:

No encontrado

Mientras:

403

significa:

Existe, pero no tienes permiso.

Para este proyecto podemos utilizar cualquiera de las dos estrategias.

18. Crear una página personalizada para error 403

Crear:

templates/403.html

Ejemplo:

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<div class="alert alert-danger">
```

```
    <h1>
```

```
        Acceso denegado
```

```
    </h1>
```

```
    <p>
```

```
        No tienes permiso para realizar  
        esta operación.
```

```
    </p>
```

```
</div>
```

```
<a
```

```
    href="{% url 'home' %}"
```

```
    class="btn btn-primary"
```

```
>
```

```
    Volver al inicio
```

```
</a>
```

```
{% endblock %}
```

19. Crear una página personalizada para error 404

Crear:

templates/404.html

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<div class="text-center py-5">
```

```
    <h1 class="display-1">
```

```
        404
```

```
    </h1>
```

```
    <h2>
```

```
        Página no encontrada
```

```
    </h2>
```

```
    <p class="text-muted">
```

```
        El recurso solicitado no existe  
        o no está disponible.
```

```
    </p>
```

```
    <a
```

```
        href="{% url 'home' %}"
```

```
        class="btn btn-primary"
```

```
    >
```

```
        Volver al inicio
```

```
    </a>
```

```
</div>
```

```
{% endblock %}
```

20. Mejorar el administrador

Podemos mejorar:

productos/admin.py

Código:

```
from django.contrib import admin
```

```
from .models import (  
    Categoria,  
    Producto,  
)
```

```
@admin.register(Categoria)  
class CategoriaAdmin(admin.ModelAdmin):
```

```
    list_display = (  
        "id",  
        "nombre",  
        "fecha_creacion",  
    )
```

```
    search_fields = (  
        "nombre",  
    )
```

```
@admin.register(Producto)  
class ProductoAdmin(admin.ModelAdmin):
```

```
    list_display = (  
        "id",  
        "nombre",  
        "categoria",  
        "usuario",  
        "precio",  
        "fecha_creacion",  
    )
```

```
    list_filter = (  
        "categoria",  
        "usuario",  
    )
```

```
    search_fields = (  
        "nombre",  
        "usuario__username",  
    )
```

```
"categoria__nombre",  
)
```

Esto permite buscar y filtrar productos desde </admin/>.

21. Mostrar número de productos del usuario

En la página principal podemos mostrar:

```
{% if user.is_authenticated %}
```

```
<p>
```

```
    Has creado:
```

```
    <strong>
```

```
        {{ user.productos.count }}
```

```
    </strong>
```

```
    producto(s).
```

```
</p>
```

```
{% endif %}
```

Esto funciona gracias a:

```
related_name="productos"
```

del modelo:

```
usuario = models.ForeignKey(  
    User,  
    on_delete=models.CASCADE,  
    related_name="productos"  
)
```

Podemos hacer:

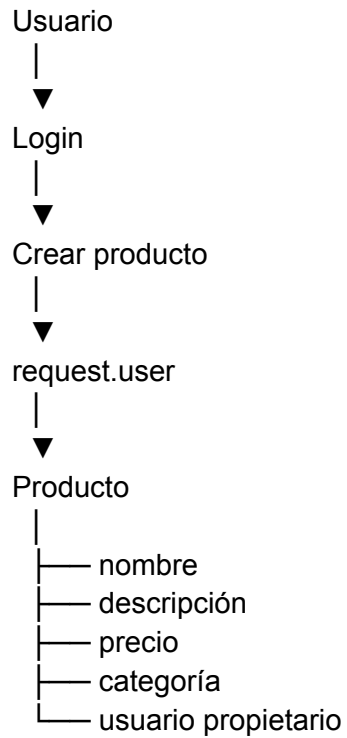
```
request.user.productos.all()
```

o:

```
request.user.productos.count()
```

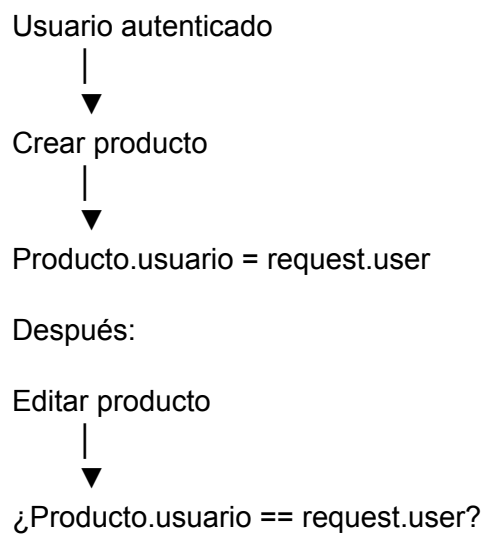
22. Flujo actualizado

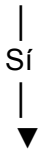
Ahora el flujo será:



23. Control de acceso

La aplicación deberá cumplir:





Permitir edición

Si no:

Acceso denegado

24. Pruebas recomendadas

Crear dos usuarios:

juan
maria

Prueba 1

Iniciar sesión como:

juan

Crear:

Producto:
Portátil

El propietario debe ser:

juan

Prueba 2

Cerrar sesión.

Entrar como:

maria

Crear:

Producto:
Monitor

El propietario será:

maria

Prueba 3

Como María entrar en:

Mis productos

Debe aparecer:

Monitor

No debe aparecer:

Portátil

Prueba 4

María puede ver el detalle público del portátil de Juan.

Pero no debe poder editarlo.

Prueba 5

María intenta acceder manualmente a:

/productos/1/editar/

si ese producto pertenece a Juan.

La edición debe quedar bloqueada.

Prueba 6

María intenta:

/productos/1/eliminar/

Tampoco debe poder eliminarlo.

25. Conceptos que se trabajan

Con esta ampliación se trabajan tres conceptos diferentes.

Autenticación

Pregunta:

¿Quién eres?

Ejemplo:

`request.user`

Autorización

Pregunta:

¿Tienes permiso para hacer esto?

Ejemplo:

`producto.usuario == request.user`

Propiedad

Pregunta:

¿A quién pertenece este recurso?

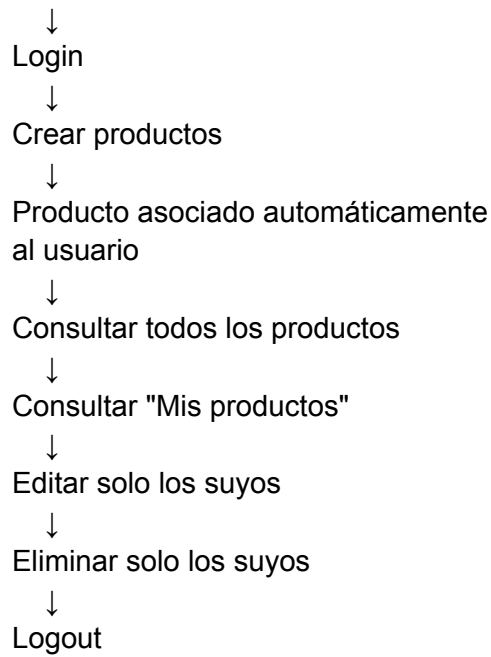
Ejemplo:

`Producto.usuario`

26. Resultado final

Ahora cada usuario podrá:

Registrarse



Y seguirá manteniendo el sistema anterior de:

Login
Registro
Logout
Reset de contraseña
Email
Token
Nueva contraseña
Repetición
Actualización de password

Con esta ampliación, el proyecto trabaja tanto **autenticación** como **autorización y propiedad de recursos**, acercándolo mucho más al funcionamiento real de una aplicación web profesional con Django.